

Timetable Management System Project Report

COURSE : DATABASE SYSTEMS (CS-3006)

INSTRUCTOR : Talha Shahid

GROUP MEMBERS:

Muhammad Bin Ismail (24K-0888)

Safiullah Shaikh (24K-0926)

Jawad Ali (24K-0907)

1. Introduction

This report documents the design, development, and implementation of the Timetable Management System — a full-stack academic scheduling application built as part of the Database Systems course at FAST NUCES. The primary goal was to create a system that solves real scheduling problems using proper database engineering, rather than pushing all logic into application code.

Academic institutions face recurring challenges when building timetables: rooms get double-booked, teachers get assigned to overlapping slots, and there is rarely a clear record of who made which changes. Our system addresses all of these issues at the database layer, which means the rules hold regardless of how the data is accessed.

The system manages six core entities — departments, teachers, courses, rooms, time slots, and semesters — and provides a scheduling interface, reporting tools, and a conflict detection mechanism.

2. System Overview

2.1 Technology Stack

Frontend: React 18 with Tailwind CSS for a responsive single-page application.

Backend: Node.js 20 with Express 4 providing a RESTful API.

Database: MySQL 8.0 accessed through the mysql2/promise driver with raw SQL.

No ORM: We deliberately avoided ORMs like Sequelize so every query, join, and procedure call is explicit and visible.

2.2 Project Structure

```
Timetable_Management_System/  
  client/                React frontend (Vite + Tailwind)  
  server/  
    app.js              Express entry point  
    routes/             API route definitions  
    controllers/        Business logic handlers  
    db/                 MySQL connection pool  
    sql/                All SQL files (tables, views, triggers, etc.)
```

3. Database Design

3.1 Schema Overview

The schema was designed following third normal form. Each entity has a surrogate integer primary key. All relationships are enforced through foreign key constraints. The central table is 'timetable', which references six other tables through foreign keys.

The nine tables in the schema are:

Table Name	Purpose
department	Stores faculty departments
teacher	Teacher records linked to a department
course	Course records with credit hours and department
teacher_course	Junction table for teacher-course many-to-many relationship
room	Room details including type (lecture/lab) and capacity
time_slot	Available time blocks per day of the week
semester	Semester records linked to departments
timetable	Main scheduling table linking all entities
timetable_audit	Audit log storing JSON snapshots of every change

3.2 Key Design Decisions

- Surrogate integer primary keys were used for all main entities for clean join performance.
- The teacher_course junction table handles the many-to-many relationship between teachers and courses, with ON DELETE CASCADE so that removing a teacher or course cleans up the mapping automatically.
- CHECK constraints enforce domain validity — credit_hours must be between 1 and 6, and end_time must be greater than start_time in time_slot.
- Unique constraints prevent logical duplicates — for example, uq_slot prevents two identical time slots, and uq_semester_course_slot prevents the same course from being scheduled twice in the same semester and slot.

3.3 Referential Integrity

Foreign key constraints use ON UPDATE CASCADE so that if a primary key changes, all referencing tables update automatically. ON DELETE RESTRICT protects parent records — you cannot delete a teacher or room that is currently referenced by a timetable entry. ON DELETE CASCADE is used selectively on the teacher_course junction table.

This design ensures data consistency even if someone accesses the database directly without going through the application.

3.4 Indexes

Three indexes were added to the timetable table on room_id, slot_id, and teacher_id. These columns appear in almost every scheduling query and join. The indexes significantly speed up conflict checks and report generation.

4. Advanced Database Features

4.1 Views

Three SQL views were created to encapsulate complex queries that are needed repeatedly across the application.

vw_full_timetable

This view joins seven tables — timetable, course, teacher, room, time_slot, semester, and department — into a single denormalized result set. Every timetable display and filter in the application reads from this view rather than writing the same multi-table join repeatedly.

vw_teacher_workload

This view groups timetable entries by teacher and counts the total number of classes assigned to each. It is used by the Reports page to show workload distribution across all teachers.

vw_room_utilization

This view calculates the total occupied hours per room by summing the duration of each time slot where a room appears in the timetable. The TIMESTAMPDIFF function computes the minutes between start_time and end_time, which is then converted to hours.

4.2 Stored Procedures

Three stored procedures were written to centralize common queries and reduce duplication in the backend code.

- sp_get_timetable_by_semester(p_semester_id) — queries vw_full_timetable filtered by semester, ordered by day and time.
- sp_get_teacher_schedule(p_teacher_id) — queries vw_full_timetable filtered by teacher, ordered by day and time.
- sp_get_free_rooms(p_slot_id) — returns all rooms not currently booked in the given time slot using a NOT IN subquery.

These procedures are called directly from the Express controllers using CALL statements, making the backend code clean and consistent.

4.3 Triggers

Four triggers are the most critical part of the database design. They run automatically and cannot be bypassed by the application.

trg_prevent_room_conflict (BEFORE INSERT)

Before any new row is inserted into timetable, this trigger checks whether the same room is already booked in the same time slot. If a match exists, it raises a SIGNAL with SQLSTATE 45000 and the message 'Room is already booked for this slot', which causes the insert to fail immediately.

trg_prevent_teacher_conflict (BEFORE INSERT)

Similarly, this trigger checks whether the same teacher already has a class in the same time slot. If so, the insert is rejected with 'Teacher already has a class in this slot'. This ensures a teacher can never be scheduled in two places at once.

trg_log_timetable_changes_insert (AFTER INSERT)

After a successful insert into the timetable table, this trigger writes a record to timetable_audit. It stores the operation type as INSERT, the current timestamp, and a JSON snapshot of the new row's course_id, teacher_id, room_id, slot_id, and semester_id.

trg_log_timetable_changes_update (AFTER UPDATE)

After any update to a timetable row, this trigger writes a record to timetable_audit with both the old data and new data as separate JSON snapshots. This makes it possible to see exactly what changed and when.

4.4 Transactions

Timetable inserts in the backend use explicit database transactions. The connection calls beginTransaction() before the INSERT, and commit() only if the insert succeeds. If any error occurs — including a trigger rejection — rollback() is called to undo any partial changes. This guarantees that the timetable is never left in an inconsistent state.

4.5 Audit Trail

The timetable_audit table provides a full history of scheduling changes. Each record stores the entry_id that was affected, the action type (INSERT or UPDATE), the timestamp, and JSON snapshots of the old and new data. This means administrators can trace any scheduling decision back to when it was made, which is valuable for accountability and debugging.

5. Backend Implementation

5.1 API Structure

The Express backend exposes three groups of API endpoints under /api/:

- CRUD routes for all master entities: departments, teachers, courses, rooms, time slots, and semesters.
- Timetable routes: GET all entries, POST new entry (with transaction), DELETE entry, plus filtered views by semester and teacher using stored procedures.
- Report routes: workload, room utilization, conflicts, unassigned courses, max workload teacher, available rooms by day/time, and weekly schedule by department.

5.2 Error Handling

SQL errors raised by triggers are mapped to HTTP 409 Conflict responses with the trigger's message text passed directly to the client. This is what allows the frontend to display meaningful error messages like 'Room is already booked for this slot' rather than a generic server error.

6. Frontend Implementation

The React frontend is a single-page application with six pages accessible from a sidebar navigation:

Page	Description
Dashboard	Summary cards showing live counts from the database
Timetable View	Weekly grid layout displaying all scheduled entries with delete buttons
Add Timetable Entry	Form with dropdowns and real-time conflict warning before submission
Manage Data	Tabbed interface for adding and deleting teachers, courses, rooms, and time slots
Reports	Teacher workload and room utilization powered by SQL views
Conflict Checker	Scans the timetable for any hidden scheduling overlaps

The frontend uses `fetch()` to call the backend API. All state is managed with React's `useState` and `useMemo` hooks. The real-time conflict warning on the Add Timetable Entry page works by checking the locally loaded timetable data before the form is even submitted, giving users an early heads-up.

7. End-to-End System Flow

The typical flow through the system works as follows:

1. An administrator opens the app and uses Manage Data to set up departments, teachers, courses, rooms, and time slots.
2. They navigate to Add Timetable Entry and select a course, teacher, room, time slot, and semester.
3. If a conflict is detected on the frontend, a warning appears immediately before any database call is made.
4. When the form is submitted, the backend begins a database transaction and runs the INSERT statement.
5. The two BEFORE INSERT triggers fire. If either detects a conflict, they raise a SIGNAL and the insert fails. The transaction rolls back.
6. If no conflict exists, the insert succeeds, the transaction commits, and the AFTER INSERT trigger logs the change to `timetable_audit`.
7. The frontend reloads the timetable data and the new entry appears in the weekly grid.

8. Work Division

Muhammad Bin Ismael — Database Layer

- Designed the complete 9-table schema with all constraints, indexes, and relationships.
- Wrote all SQL files: tables, views, stored procedures, triggers, and seed data.
- Implemented the four triggers for conflict prevention and audit logging.
- Created the three SQL views for timetable display and reporting.
- Wrote the three stored procedures for filtered data retrieval.
- Responsible for database setup, MySQL Workbench demonstration, and explaining all DB-level features during the viva.

Safiullah Shaikh — Application Layer

- Built the Node.js Express backend with all API routes and controllers.
- Implemented transactional timetable inserts with rollback logic.
- Built the React frontend with all six pages including forms, tables, and the weekly grid.
- Integrated the frontend with all backend API endpoints.
- Responsible for demonstrating the full application flow during the viva.

9. Testing and Verification

9.1 Trigger Testing

We verified both conflict triggers by attempting to insert a timetable entry where the room was already booked and where the teacher was already scheduled. In both cases the trigger fired, the SIGNAL was raised, the transaction rolled back, and the error message appeared correctly in the frontend.

9.2 Audit Log Verification

After each successful timetable insertion, we queried the timetable_audit table directly in MySQL Workbench and confirmed that a new row was added with the correct entry_id, action type, timestamp, and JSON snapshot.

9.3 Report Verification

We confirmed teacher workload counts manually by counting timetable entries per teacher and comparing with the view output. Room utilization hours were verified by calculating expected occupied time from the seed data time slots.

10. Conclusion

The Timetable Management System successfully demonstrates the practical application of advanced database concepts in a real working application. The project goes well beyond a basic CRUD system by enforcing business rules inside the database itself through triggers, maintaining data consistency through transactions and foreign key constraints, and generating meaningful insights through SQL views.

The most important lesson from this project is that a well-designed database is not just a storage layer, it is an active participant in enforcing correctness. Our triggers ensure that no amount of buggy application code or direct database access can create a double booking. The audit trail means that every scheduling decision is permanently recorded. These are properties that would be very difficult to achieve if the logic lived only in the application.

Both team members contributed meaningfully to the project, with a clear division between the database engineering work and the application development work. The system is fully functional and ready for demonstration.